React js

Usereducer, useref, usecallback, usememo, memo, conditional rendering, controlled components, context api

Practice Tasks

- **Button (Props)**
  - Create a button that accepts `name`, `color`, and `size` as props.
  - Render multiple buttons with different styles using props.
- **Conditional Rendering**
  - Show a welcome message if `isLoggedIn` is true, otherwise show a login form.
  - Use logical AND (`&&`) to display a "Premium User" badge only for premium accounts.
- **Controlled Component**
  - Build a form with controlled inputs for `name`, `email`, and `password`.
  - Display the entered values below the form in real-time.
- **useEffect**
  - Fetch data from a public API (like JSONPlaceholder) and display it.
  - Add a cleanup function to log when the component unmounts.
- **useReducer**
  - Create a counter with increment, decrement, and reset actions.
  - Extend it to handle multiplication and division actions.
- **useRef**
  - Focus an input field when a button is clicked.
  - Track how many times a component renders using `useRef`.
- **useCallback + memo**
  - Pass a memoized function to a child component and observe re-render behavior.
  - Add another state in the parent and confirm the child doesn't re-render unnecessarily.
- **useMemo**
  - Calculate a heavy computation (like factorial or large multiplication).
  - Show how memoization avoids recalculating unless dependencies change.

- **Context (Provider + Consumer)**
  - Create a theme context with "light" and "dark".
  - Add a toggle button to switch themes globally across components.

Theory Notes

- **Props**: Used to pass data from parent to child. Immutable inside child components.
- **Conditional Rendering**: Render UI based on conditions using ternary (`? :`) or logical operators (`&&`).
- **Controlled Components**: Form inputs controlled by React state (`value` + `onChange`).
- **useEffect**: Side effects in React (API calls, subscriptions). Runs after render. Dependency array controls when it runs.
- **useReducer**: Alternative to `useState` for complex state logic. Uses `(state, action)` reducer function.
- **useRef**: Provides a way to persist values across renders without causing re-renders. Often used for DOM references.
- **useCallback**: Memoizes functions so they don't get recreated on every render. Useful when passing functions to child components.
- **useMemo**: Memoizes values (results of expensive computations).
- **Context API**: Provides global state without prop drilling. `Provider` supplies data, `Consumer` or `useContext` consumes it.
- **memo**: Higher-order component that prevents unnecessary re-renders if props haven't changed.

```
import "./App.css";
```

```
import Button from "./Button";
import Conditional from "./Conditional";
import Controlled from "./Controlled";
import Image from "./Image";
import Redmi from "./Redmi";
import Useeffects from "./Useeffects";
import Usereducer from "./Usereducer";
import Useref from "./Useref";
import Usecallback from "./Usecallback";
import Consumer from "./Context/Consumer";
import Provider from "./Context/Provider";

function Add() {
  return <h1>Hello Add component</h1>;
}

function App() {
  const data = {
    degree: "BCA",
    college: "Arts & science",
  };

  return (
    <>
      <Usecallback />
      <hr />
      <Provider>
        <Consumer />
      </Provider>
      {/* <Useeffects /> */}
      {/* <Controlled /> */}
      {/* <Usereducer /> */}
      {/* <Conditional isLoggedIn={true} /> */}
      {/* <Useref /> */}

      {/* <h1>Hello there this is react project</h1>
      <h2>Hello React </h2>
      <Add />
      <Redmi />
      <Image />
      <Image />
      <Image />
      <Image />
      <Image /> */}
      {/* <Button name="Submit" academy="fiit" />
      <Button name="Add to Cart" /> */}
      {/* <Button name="Buy Now"  details ={data}/> */}
    </>
```

```
  );
}

export default App;
```

```
import React from "react";

const Button = (props) => {
  const style = {
    backgroundColor: "black",
    color: "white",
    padding: "10px",
  };
  console.log(props.details);

  return (
    <>
      <button style={style}>
        {props.name}{" "}
      </button>
      <h1>{props.academy}</h1>
      <h2>{props.details.degree}</h2>
      <h2>{props.details.college}</h2>
    </>
  );
};

export default Button;

import React from "react";

const Conditional = ({ isLoggedIn }) => {
  console.log(isLoggedIn);

  return (
    <div>
      <h2>This is conditional rendering component</h2>

      {/* ternary operator */}
      {isLoggedIn ? (
        <h2>User logged in successfully</h2>
      ) : (
        <h2>User logged out successfully</h2>
      )}

      {/* logical operator  */}
```

```
      {/* && */}
      {isLoggedIn && <h3>This is logical AND</h3>}
    </div>
  );
};

export default Conditional;
```

```
import React, { useState } from "react";

const Controlled = () => {
  const [data, setData] = useState("");

  console.log(data);

  return (
    <div>
      <h2>This is controlled component</h2>
      <input
        type="text"
        value={data} /*controlled form */
        onChange={(e) => setData(e.target.value)}
        placeholder="Enter your name"
      />
      <input type="email" placeholder="Enter your email" />
    </div>
  );
};

export default Controlled;

import React from "react";

const Image = () => {
  return (
    <div>
      <img
        src="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcStvL8kNVKvJskGpi8do02RNw2bn3sKxTTJ2g&s"
        alt="User profile"
      />
    </div>
  );
};

export default Image;
```

```
import React, { useCallback, useMemo, useState } from "react";
import Child from "./Child";

const Usecallback = () => {
  const [count, setCount] = useState(11);
  const [other, setOther] = useState(false);
  console.log("parent component");

  const handleClick = useCallback(() => {
    console.log("handleclick function ");
  }, [count]); // depdency array

  const square = useMemo(() => {
    console.log("square function");

    return count * 100000000;
  }, [count]);

  return (
    <div>
      <h2>This is Usecallback component</h2>
      <h2>Square - {square}</h2>
      <button onClick={() => setOther(!other)}>Parent</button>
      <button onClick={() => setCount(count + 1)}>Count</button>

      <hr />
      <Child count={count} handleClick={handleClick} />
    </div>
  );
};

export default Usecallback;

import React, { memo } from "react";

const Child = memo(({ count, handleClick }) => {
  console.log("child component");

  return (
    <div>
      <h3>Child component </h3>
      <h3>{count}</h3>
      <button onClick={handleClick}>Handleclick</button>

    </div>
  );
});
```

```
export default Child;

import React, { useEffect, useState } from "react";

const Useeffects = () => {
  const [data, setData] = useState(10);
  useEffect(() => {
    console.log("Useeffect is running ");
  }, [data]);
  //   empty => renders on every changes
  // [] => render only once
//   [data]=> render when that state change
  return (
    <div>
      <h1>{data}</h1>
      <h1>Hello this is useEffects component</h1>
      <h2>Hello this is h2 element</h2>
    </div>
  );
};

export default Useeffects;

import React, { useReducer } from "react";

const Usereducer = () => {
  const addTo = (state, action) => {
    switch (action.type) {
      case "increment":
        return state + 1;
      case "decrement":
        return state - 1;
      case "reset":
        return 10;
    }
  };
  const [state, dispatch] = useReducer(addTo, 10);

  const useStyle = {
    color: "blue",
    backgroundColor: "black",
    fontSize: "30px",
  };

  return (
    <div >
      <h2
```

```
        style={{ color: "green", backgroundColor: "black", fontSize: "40px" }}
      >
        This is UseReducer component
      </h2>
      <h2 style={useStyle}>Second h2 element </h2>

      <h2>{state}</h2>
      <button onClick={() => dispatch({ type: "increment"
})}>Increment</button>
      <button onClick={() => dispatch({ type: "decrement"
})}>Decrement</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
};

export default Usereducer;

import React, { useRef } from "react";

const Useref = () => {
  const inputRef = useRef();

  const handleClick = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <h1>This is useRef component</h1>
      <input type="text" ref={inputRef} placeholder="Enter your name" />
      <button onClick={handleClick}>click to focus</button>
    </div>
  );
};

export default Useref;

import React, { createContext } from 'react'


const Store = createContext();


export default Store;
```

```
import React, { useState } from "react";
import Store from "./Store";

const Provider = ({ children }) => {
  const [theme, setTheme] = useState("light");
  return (
    <Store.Provider value={{ theme, setTheme }}>
        {children}
    </Store.Provider>
  );
};

export default Provider;
```

```
import React, { useContext } from "react";
import Store from "./Store";

const Consumer = () => {
  const { theme, setTheme } = useContext(Store);
  return (
    <div>
      <h2>{theme}</h2>
      <button onClick={() => setTheme("dark")}>Dark theme</button>
      <button onClick={() => setTheme("light")}>Light theme</button>
    </div>
  );
};

export default Consumer;
```